

Automated Testing Report

Inventory Management Project

Testing framework: pytest

Command used: python -m pytest app/tests -v

Test result:

10 tests collected

10 tests passed

0 failed

0 errors

Summary: The automated tests verify the core service logic of the inventory management system. The tests cover inventory stock movements, product creation/deletion, validation errors, and user creation including duplicate username/email checks.

Test categories:

- Inventory service tests
- Product service tests
- User service tests

Used test data: A clean in-memory SQLite database is created for each test using pytest fixtures. The base data contains one category, one storage location, one admin user, and one product.

Final result: All automated tests passed successfully.

Automated Test Case Overview

Test Case ID	Description	Expected Result	Actual Result	Status
TC_001	Stock-in increases product quantity	Quantity increases correctly	Passed	Passed
TC_002	Stock-out decreases product quantity	Quantity decreases correctly	Passed	Passed
TC_003	Stock-out more than available	ValueError is raised	Passed	Passed
TC_004	Low stock products returned	Product appears as low stock	Passed	Passed
TC_005	Create product successfully	Product is created	Passed	Passed
TC_006	Empty product name	ValueError is raised	Passed	Passed
TC_007	Delete product	Product is removed	Passed	Passed
TC_008	Create user successfully	User is created	Passed	Passed
TC_009	Duplicate email	ValueError is raised	Passed	Passed
TC_010	Duplicate username	ValueError is raised	Passed	Passed

Test Cases – Inventory Management Project

TC_001 – Stock-in increases product quantity

Field	Details
Test case ID	TC 001
Test case title/description	Verify that stock-in operation increases the product quantity correctly.
Preconditions	A clean test database is available. A product exists with quantity 10. A user exists.
Test steps	1. Create InventoryService. 2. Select existing product and user from base test data. 3. Call <code>stock_in(product.id, user.id, 3, note="Restock")</code> . 4. Check product quantity.
Test data/input	Product: Laptop, initial quantity: 10, stock-in quantity: 3, user: admin
Expected result	Product quantity increases from 10 to 13. A stock movement record is created.
Actual result	Product quantity increased correctly and movement was created.
Status	Pass
Comments	No issues found.

TC_002 – Stock-out decreases product quantity

Field	Details
Test case ID	TC 002
Test case title/description	Verify that stock-out operation decreases the product quantity correctly.
Preconditions	A clean test database is available. A product exists with quantity 10. A user exists.
Test steps	1. Create InventoryService. 2. Select existing product and user. 3. Call <code>stock_out(product.id, user.id, 4, note="Shipment")</code> . 4. Check product quantity.
Test data/input	Product: Laptop, initial quantity: 10, stock-out quantity: 4
Expected result	Product quantity decreases from 10 to 6. A stock movement record is created.
Actual result	Product quantity decreased correctly and movement was created.
Status	Pass
Comments	No issues found.

TC_003 – Stock-out more than available raises error

Field	Details
Test case ID	TC 003
Test case title/description	Verify that the system prevents stock-out if the requested quantity is greater than available stock.
Preconditions	A product exists with quantity 10.

Test steps	1. Create InventoryService. 2. Select existing product and user. 3. Call <code>stock_out(product.id, user.id, 999)</code> . 4. Check whether an error is raised.
Test data/input	Product quantity: 10, requested stock-out quantity: 999
Expected result	The system raises <code>ValueError</code> with message “Not enough stock”.
Actual result	<code>ValueError</code> was raised correctly.
Status	Pass
Comments	Edge case tested successfully.

TC_004 – Low stock products are returned correctly

Field	Details
Test case ID	TC_004
Test case title/description	Verify that products below or equal to minimum stock level are returned as low-stock products.
Preconditions	A product exists with quantity 10 and minimum quantity 2.
Test steps	1. Create InventoryService. 2. Perform stock-out of 8 units. 3. Product quantity becomes 2. 4. Call low-stock check function.
Test data/input	Product: Laptop, initial quantity: 10, stock-out quantity: 8, minimum quantity: 2
Expected result	Product is included in the low-stock product list.
Actual result	Product was returned as expected.
Status	Pass
Comments	Low-stock logic works correctly.

TC_005 – Create product successfully

Field	Details
Test case ID	TC_005
Test case title/description	Verify that a new product can be created with valid input data.
Preconditions	A valid category and storage location exist in the database.
Test steps	1. Create ProductService. 2. Call <code>create_product()</code> with valid product data. 3. Save product to database. 4. Verify created product fields.
Test data/input	Name: Keyboard, description: Mechanical keyboard, quantity: 5, minimum quantity: 1
Expected result	Product is created successfully and stored in the database.
Actual result	Product was created successfully.
Status	Pass
Comments	Happy path test successful.

TC_006 – Create product with empty name raises error

Field	Details
Test case ID	TC_006
Test case title/description	Verify that the system rejects product creation with an empty name.

Preconditions	ProductService is available.
Test steps	1. Create ProductService. 2. Call <code>create_product()</code> with empty product name. 3. Check whether an error is raised.
Test data/input	Product name: empty string
Expected result	The system raises <code>ValueError</code> with message "Product name cannot be empty".
Actual result	<code>ValueError</code> was raised correctly.
Status	Pass
Comments	Validation works correctly.

TC_007 – Delete product removes record

Field	Details
Test case ID	TC_007
Test case title/description	Verify that an existing product can be deleted from the database.
Preconditions	A product exists in the database.
Test steps	1. Create ProductService. 2. Select existing product ID. 3. Call <code>delete_product(product_id)</code> . 4. Try to retrieve the deleted product.
Test data/input	Existing product: Laptop
Expected result	Product is removed and cannot be found afterwards.
Actual result	Product was deleted successfully.
Status	Pass
Comments	Delete operation works correctly.

TC_008 – Create user successfully

Field	Details
Test case ID	TC_008
Test case title/description	Verify that a new user can be created with valid username and email.
Preconditions	UserService is available. No user with same username or email exists.
Test steps	1. Create UserService. 2. Call <code>create_user("employee1", "employee1@example.com")</code> . 3. Verify created user fields.
Test data/input	Username: employee1, email: employee1@example.com
Expected result	User is created successfully.
Actual result	User was created successfully.
Status	Pass
Comments	User creation works correctly.

TC_009 – Duplicate email raises error

Field	Details
Test case ID	TC_009
Test case title/description	Verify that the system prevents creating two users with the same email address.

Preconditions	A user with email <code>employee1@example.com</code> already exists.
Test steps	1. Create first user. 2. Try to create second user with same email. 3. Check whether an error is raised.
Test data/input	Existing email: employee1@example.com
Expected result	The system raises <code>ValueError</code> because email already exists.
Actual result	<code>ValueError</code> was raised correctly.
Status	Pass
Comments	Duplicate email validation works.

TC_010 – Duplicate username raises error

Field	Details
Test case ID	TC_010
Test case title/description	Verify that the system prevents creating two users with the same username.
Preconditions	A user with username <code>employee1</code> already exists.
Test steps	1. Create first user. 2. Try to create second user with same username. 3. Check whether an error is raised.
Test data/input	Existing username: <code>employee1</code>
Expected result	The system raises <code>ValueError</code> because username already exists.
Actual result	<code>ValueError</code> was raised correctly.
Status	Pass
Comments	Duplicate username validation works.

Test Execution Result

```

1 file changed, 3 insertions(+), 3 deletions(-)
PS C:\Users\aadat\Desktop\inventory-management-fhnw> python -m pytest app/tests -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\aadat\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\aadat\Desktop\inventory-management-fhnw
configfile: pytest.ini
plugins: anyio-4.12.1, cov-7.1.0
collected 10 items

app/tests/test_inventory_service.py::test_stock_in_increases_quantity PASSED [ 10%]
app/tests/test_inventory_service.py::test_stock_out_decreases_quantity PASSED [ 20%]
app/tests/test_inventory_service.py::test_stock_out_more_than_available_raises_error PASSED [ 30%]
app/tests/test_inventory_service.py::test_get_low_stock_products_returns_expected_items PASSED [ 40%]
app/tests/test_product_service.py::test_create_product_success PASSED [ 50%]
app/tests/test_product_service.py::test_create_product_with_empty_name_raises_error PASSED [ 60%]
app/tests/test_product_service.py::test_delete_product_removes_record PASSED [ 70%]
app/tests/test_user_service.py::test_create_user_success PASSED [ 80%]
app/tests/test_user_service.py::test_create_user_duplicate_email_raises_error PASSED [ 90%]
app/tests/test_user_service.py::test_create_user_duplicate_username_raises_error PASSED [100%]

===== 10 passed in 0.20s =====
PS C:\Users\aadat\Desktop\inventory-management-fhnw>

```

All automated tests were executed successfully using pytest.